

CLAUDE.md frente a hooks para formato y normas de empresa

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES Sesión 6 de 12 · Notas en español (es-ES)

La sesión 6 (13 de agosto de 2026) dejó el contraste que las respuestas de Foundation siguen mezclando: **CLAUDE.md** (y las skills) dicen al *modelo* cómo escribir código; los **hooks** corren *en local* en el ciclo de vida de Claude Code y no consumen tokens de generación. El ejemplo en directo fue Prettier vía un hook post-tool-use tras un Edit/Write, frente a meter «formatea como Prettier / estilo de empresa» en la memoria para que el LLM reescriba el fichero en un segundo pase de pago.

LOS NOMBRES DE EVENTOS DE HOOK EN LOS DOCS PUEDEN DIFERIR

El formador listó pre-tool use, post-tool use, user prompt submit, session start/end, stop, sub-agent stop, notification y pre-compact. Confirmad el catálogo vigente en la [documentación de Claude Code](#). El escape de comillas en `settings.json` y si el chat del IDE frente al terminal dispara de verdad los hooks también bailó en el demo (hubo que activar un ajuste preview «Claude hook»).

Dos formas de obtener el «formato de empresa»

Enfoque	Dónde vive	Quién lo ejecuta	Tokens	Mejor para
CLAUDE.md / skills	Markdown en el repo (siempre activo o slash)	El modelo, como instrucciones extra en cada petición	Sí — las reglas de formato se releen y a menudo se reaplican como generación	Arquitectura, elección de stack, SRP, «usad Mermaid», reglas de dominio que el modelo debe entender
Hook (comando en settings.json)	Ajustes de Claude Code + scripts en el proyecto	Shell local/sandbox antes o después de una tool	No — NPX Prettier en disco no es un turno de LLM	Formato mecánico, lint --fix, bloquear rm, proteger .env

La «situación ideal» del formador: *no* pidáis al LLM que haga pretty-print. Usad hooks como plugin extendido del espacio de trabajo. Pregunta de Lalit — «los hooks no cuentan como uso de tokens, ¿verdad?» — respuesta: sí, porque NPX (o cualquier shell) corre en el sistema local / sandbox, no como salida del modelo.

Qué hizo de verdad un formateador post-tool-use

Prompt: añade un mensaje hello en la página de login del frontend. Claude invocó Edit/Write (tools de IO). Tras la escritura, corrió el hook **post-tool-use**: echo de estado (visible en la sesión Claude de terminal — la GUI no tenía un log dedicado de hooks) y `npx prettier` sobre el fichero tocado. Diff: comillas simples a dobles, JSX largo partido en varias líneas — mecánica de código limpio, no comportamiento nuevo de

producto. Las opciones de Prettier (puntos y coma, trailing commas, indentación) van en la config de Prettier que consume el hook, no en un párrafo de CLAUDE.md.

- El cuerpo del hook son comandos Linux/macOS (y lo que el shell del sandbox pueda ejecutar). Las comillas dobles en JSON hay que escaparlas; un mal quoting fue por lo que el hook del día anterior no disparó.
- Las cadenas de estado que incrustáis aparecen en la UI de ejecución de tools; no son un protocolo JSON especial salvo que emitáis JSON a propósito.
- Verificad los hooks desde Claude en **terminal** si la superficie de chat de VS Code calla: los compañeros vieron disparo solo en terminal hasta activar un ajuste preview.

Dejad CLAUDE.md para lo que Prettier no puede saber

La memoria sigue ganando para: valores por defecto listos para producción, qué framework, «controladores antes que servicios», Mermaid y no JPEG, ruta OpenAPI, no inventar campos. Eso exige que el modelo *elija* distinto. El formato, excluir ficheros secretos y «nunca ejecutes este bash» van mejor como hooks porque son deterministas y baratos. Meter ambas cosas en CLAUDE.md hace que el modelo narre pases de formato que podríais haber tenido gratis.

Ciclo de vida: los hooks son eventos, no skills

Las skills siguen inyectando roles al hacer slash. Los hooks disparan en el flujo de actividad de Claude da igual qué skill se usó. Session start/end, prompt submit, compact, sub-agent stop: cada uno puede lanzar un comando. Así se «gestiona el ciclo de vida de la interacción de Claude con los LLM», en la frase del formador. Se configuran en `settings.json` con `matchers` (qué tool: Edit, Write, Bash, ...).

CLAUDE.MD NO PUEDE DETENER EL BASH EN MODO AUTO

En modo auto/bypass el modelo no preguntará antes de un `rm`. Una frase en CLAUDE.md («nunca borréis ficheros») es una sugerencia. Un matcher **pre-tool-use** que ejecute `block-dangerous-commands.sh` puede denegar la tool. Ese emparejamiento es el capítulo siguiente; la frase de examen de este es: el formato y la política que deben ocurrir siempre van en hooks, no en prosa de memoria.

Comprobación de estudio

Escenario escrito: «la guía de estilo del equipo es indentación de 2 espacios y comillas dobles; además deben usar el patrón repositorio». Partid la respuesta: indentación/comillas → hook post-tool de Prettier; patrón repositorio → CLAUDE.md de backend o una skill de backend. Mezclarlo todo en un solo CLAUDE.md es el distractor.

Documentación oficial

- **Visión general de Claude Code**

<https://code.claude.com/docs/en/overview>

Hooks, settings.json y su relación con CLAUDE.md.

- **Inicio de Claude Docs**

<https://docs.claude.com/en/home>

Si los nombres de eventos de hook se renombraron tras agosto de 2026.

- **Guardrails**

<https://docs.claude.com/en/docs/test-and-evaluate/strengthen-guardrails>

Las comprobaciones deterministas van fuera del modelo cuando sea posible.

- **Visión general de prompting**

<https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>

Por qué las instrucciones extra de formato en el prompt duplican de forma cara un hook.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.